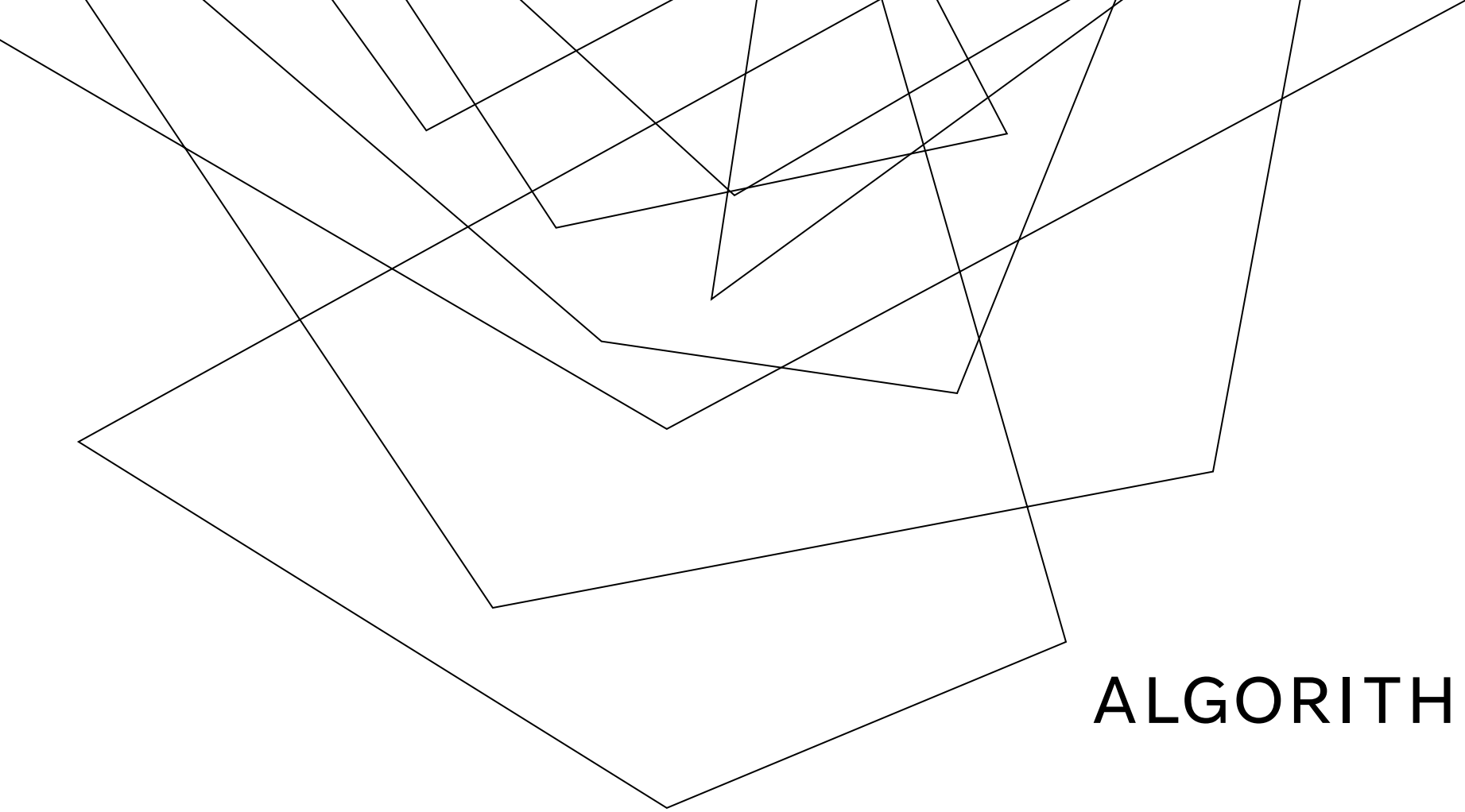


A series of thin, black, overlapping geometric lines and polygons in the top-left corner of the slide, creating a complex, abstract pattern.

ALGORITHMS

2. ALGORITHM ANALYSIS





목차

목차

1. Algorithm analysis
2. Best / Worse / Average case
3. Asymptotic analysis



An abstract geometric design featuring two thin, dark lines that intersect on a light gray background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is steeper, running from the top-center towards the bottom-right. The intersection point is located to the left of the text.

ALGORITHM ANALYSIS

ALGORITHM

- 알고리즘 : 어떤 문제 (Problem) 를 해결하기 위한 절차나 방법을 공식화한 형태로 표현한 것
- 문제 : 입력 (Input) 과 출력 (Output) 의 관계

Ex1) 정렬 문제

- 입력 :
- 출력 :

Ex2) 최단경로 문제

- 입력 :
- 출력 :



ALGORITHM EFFICIENCY

- 한 문제는 다양한 알고리즘으로 해결할 수 있음

Ex) 정수 n 을 제공하는 문제

- 입력 : 정수 n
- 출력 : n 이 n 번 더해진 값

Algorithm 1

```
sum = 0
for i in range(0, n):
    sum = sum + n
```

Algorithm 2

```
sum = 0
for i in range(0, n):
    for j in range(0, n):
        sum = sum + 1
```

Algorithm 3

```
sum = n * n
```



ALGORITHM EFFICIENCY

- 한 문제는 다양한 알고리즘으로 해결할 수 있음

Ex) 정수 n 을 제공하는 문제

- 입력 : 정수 n
- 출력 : n 이 n 번 더해진 값

Algorithm 1

```
sum = 0
for i in range(0, n):
    sum = sum + n
```

Algorithm 2

```
sum = 0
for i in range(0, n):
    for j in range(0, n):
        sum = sum + 1
```

Algorithm 3

```
sum = n * n
```

➔ Q) 어떤 알고리즘이 “좋은” 알고리즘일까?



ALGORITHM EFFICIENCY

- “좋은” 알고리즘 = 한정된 자원, 적은 자원으로 문제를 해결할 수 있는
“효율적인” 알고리즘
- 자원의 종류
 - 시간(Time) : 실행 시간 → 본 수업에서 주로 다룰 예정
 - 공간(Space) : 메모리 사용량



EFFICIENCY MEASUREMENT

- $T(n)$: 입력 n 에 대한 시간 복잡도 함수 (= 기본 연산의 수)
 - 기본 연산 : 사칙연산(+, -, *, /), 대입연산(=), 비교연산(<, >, ==) 등

Ex) Algorithm 1

```
sum = 0
for i in range(0, n):
    sum = sum + n
```

Algorithm 2

```
sum = 0
for i in range(0, n):
    for j in range(0, n):
        sum = sum + 1
```

Algorithm 3

```
sum = n * n
```

Q) For문 loop 내의 변수 증가 ($i++$)도 기본 연산 수에 포함?

A) 기본 연산은 문제의 해결에 직접 기여하는 연산을 의미

따라서 For문 loop 내의 변수 증가는 카운팅하지 x



EFFICIENCY MEASUREMENT

- $T(n)$: 입력 n 에 대한 시간 복잡도 함수 (= 기본 연산의 수)
 - 기본 연산 : 사칙연산(+, -, /, *), 대입연산(=), 비교연산(<, >, ==) 등

Ex) Algorithm 1

```
sum = 0
for i in range(0, n):
    sum = sum + n
```

Algorithm 2

```
sum = 0
for i in range(0, n):
    for j in range(0, n):
        sum = sum + 1
```

Algorithm 3

```
sum = n * n
```

	Algorithm 1	Algorithm 2	Algorithm 3
대입연산			
덧셈연산			
곱셈연산			
합계			





BEST / WORSE /
AVERAGE CASE

BEST / WORST / AVERAGE CASE

- 입력 크기가 같아도 입력에 따라 복잡도가 달라지는 문제도 존재함

Ex) n개의 숫자를 담은 배열에서 주어진 값의 인덱스를 찾는 문제

- 입력 : 배열 nums, 찾고 싶은 값 q
- 출력 : q의 인덱스 혹은 -1

3	1	8	7	2	5	4	6
---	---	---	---	---	---	---	---

```
def sequential_search(nums, q):  
    for i, x in enumerate(nums):  
        if x == q: return i  
    return -1
```

- Best case : $T(n) =$
- Worst case : $T(n) =$
- Average case : $T(n) =$



An abstract geometric design featuring two thin, dark gray lines that intersect on a light gray background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is oriented from the top-right towards the bottom-left. The intersection point is located to the left of the text.

ASYMPTOTIC ANALYSIS

ASYMPTOTIC ANALYSIS

- n 이 아주 커지면 시간 복잡도 $\tau(n)$ 은 어떤 양상을 보일까?

as·ymp·tot·ic

발음 미국식 [æˈsɪmptətɪk(əl)] 영국식 [ˈtɒs-]



부표제어 asymptotical

동아출판

YBM

교학사

형용사 | 영영사전

형용사

1. 수학

점근선의

asymptotic property



점근성

점근선

문서 토론

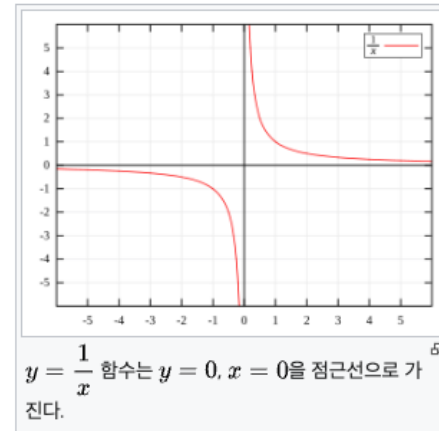
읽기 편집 역사 보기 도구

위키백과, 우리 모두의 백과사전.

점근선(漸近線, 영어: asymptote)은 무한히 뻗어나가는 곡선에서 곡선 위의 동점이 원점에서 멀어질 때, 그 점에서 어떤 정해진 직선과의 거리가 0으로 수렴해 갈 때, 그 정해진 선이다. [해석기하학](#), [사영기하학](#) 등에서 사용된다.^{[1][2]}

‘asymptote’이라는 단어는 그리스어 ἀσύμπτωτος(*asumptōtos*)에서 유래했는데 이는 ‘함께 떨어지지 않다’라는 의미이다.^[3] 여기서 ἀ는 부정 접두사, σύν은 ‘함께’, πτωτ-ός는 ‘떨어진’이라는 의미를 지닌다. 이 용어는 [페르게의 아폴로니오스](#)가 [원뿔 곡선](#)에 관한 저서에서 처음 도입했으나, 현대의 의미와 달리 그는 주어진 곡선과 교차하지 않는 모든 선을 의미하는 용어로 사용했다.^[4]

점근선에는 수평 점근선, 수직 점근선, 사선 점근선의 세 종류가 있다. 함수 $y = f(x)$ 의 그래프로 주어진 곡선의 경우, 수평 점근선은 x 가 $+\infty$ 또는 $-\infty$ 로 갈 때 함수 그래프가 가까워지는 수평선이다. 수직 점근선은 함수가 무한대로 발산하는 수직선이다. 사선 점근선은 기울기가 0이 아니면서 유한한 직선으로, x 가 $+\infty$ 또는 $-\infty$ 로 갈 때 그래프가 그 직선에 가까워진다.



ASYMPTOTIC ANALYSIS

- n 이 아주 커지면 시간 복잡도 $\tau(n)$ 은 어떤 양상을 보일까?

as·ymp·tot·ic

발음 미국식 [æˈsɪmtəˈtɪk(əl)] 영국식 [ˈtɒs-]



부표제어 asymptotical

동아출판

YBM

교학사

형용사 | 영영사전

형용사

1. 수학

점근선의

asymptotic property



점근성

점근선

문서 토론

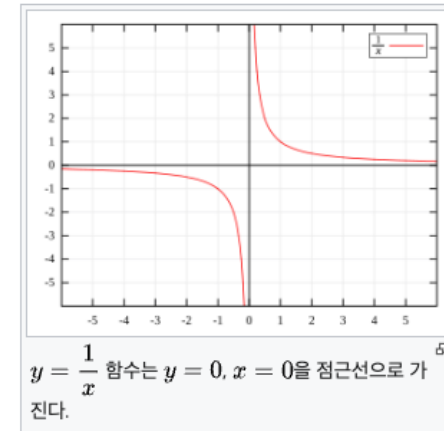
읽기 편집 역사 보기 도구

위키백과, 우리 모두의 백과사전.

점근선(漸近線, 영어: asymptote)은 무한히 뻗어나가는 곡선에서 곡선 위의 동점이 원점에서 멀어질 때, 그 점에서 어떤 정해진 직선과의 거리가 0으로 수렴해 갈 때, 그 정해진 선이다. [해석기하학](#), [사영기하학](#) 등에서 사용된다.^{[1][2]}

'asymptote'이라는 단어는 그리스어 ἀσύμπτωτος(asumptōtos)에서 유래했는데 이는 '함께 떨어지지 않다'라는 의미이다.^[3] 여기서 ἀ는 부정 접두사, σύν은 '함께', πτωτ-ός는 '떨어진'이라는 의미를 지닌다. 이 용어는 [페르게의 아폴로니오스](#)가 [원뿔 곡선](#)에 관한 저서에서 처음 도입했으나, 현대의 의미와 달리 그는 주어진 곡선과 교차하지 않는 모든 선을 의미하는 용어로 사용했다.^[4]

점근선에는 수평 점근선, 수직 점근선, 사선 점근선의 세 종류가 있다. 함수 $y = f(x)$ 의 그래프로 주어진 곡선의 경우, 수평 점근선은 x 가 $+\infty$ 또는 $-\infty$ 로 갈 때 함수 그래프가 가까워지는 수평선이다. 수직 점근선은 함수가 무한대로 발산하는 수직선이다. 사선 점근선은 기울기가 0이 아니면서 유한한 직선으로, x 가 $+\infty$ 또는 $-\infty$ 로 갈 때 그래프가 그 직선에 가까워진다.



➔ Q) 왜 이런 분석을 할까?



ASYMPTOTIC ANALYSIS

- 여러 항으로 구성된 $T(n)$ 의 점근분석 예시

Ex) $T(n) = n^2 + n + 1$

n	T(n)
n = 1	$T(n) = 1 + 1 + 1 = 3$ (33.3% for n^2)
n = 10	$T(n) = 100 + 10 + 1 = 111$ (90% for n^2)
n = 100	$T(n) = 10000 + 100 + 1 = 10101$ (99% for n^2)
n = 1000	$T(n) = 1000000 + 1000 + 1 = 1001001$ (99.9%)

➔ $T(n)$ 에서 제일 차수 높은 우세항만 보면 됨 & 상수계수는 무시해도 됨



ASYMPTOTIC ANALYSIS

Revisit)

Algorithm 1

```
sum = 0
for i in range(0, n):
    sum = sum + n
```

Algorithm 2

```
sum = 0
for i in range(0, n):
    for j in range(0, n):
        sum = sum + 1
```

Algorithm 3

```
sum = n * n
```

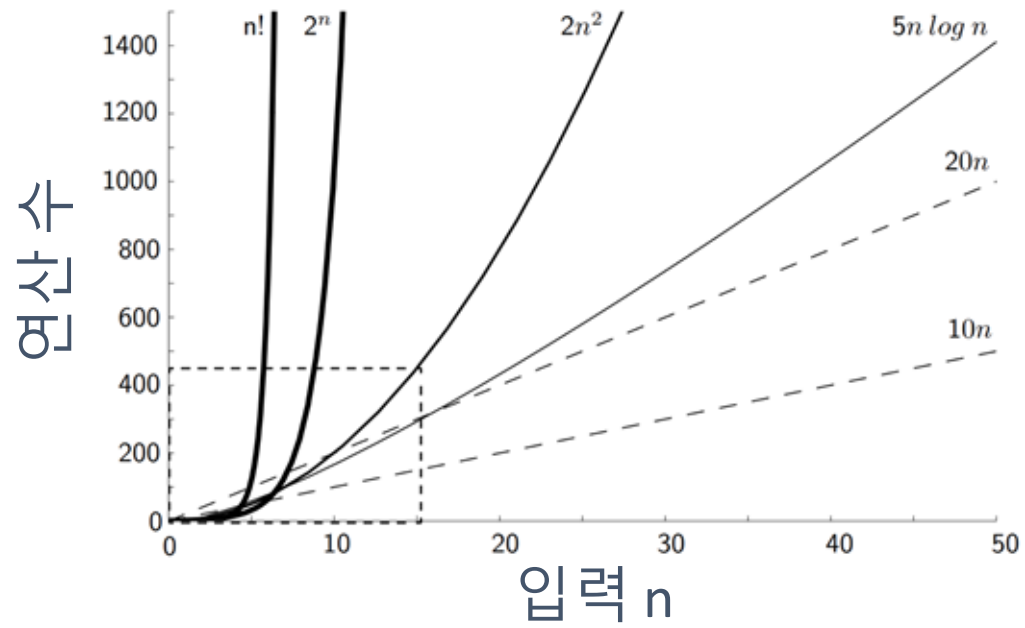
	Algorithm 1	Algorithm 2	Algorithm 3
기본 연산 수	$2n+1$	$2n^2+1$	2
점근적 분석			



ASYMPTOTIC ORDER

- 복잡도 함수의 서열

$\log n$	$<$	n	$<$	$n \log n$	$<$	n^2	$<$	n^3	$<$	2^n
0		1		0		1		1		2
1		2		2		4		8		4
2		4		8		16		64		16
3		8		24		64		512		256
4		16		64		256		4096		65536
5		32		160		1024		32768		4294967296



ASYMPTOTIC ORDER

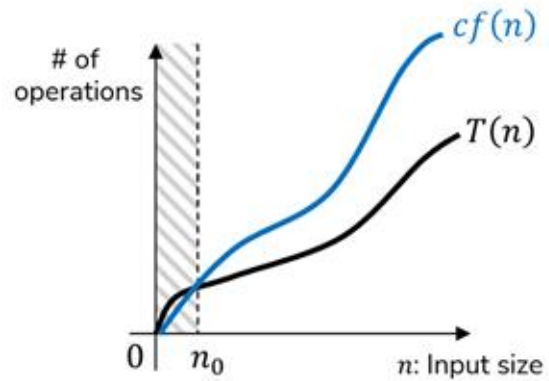
- 빨리 증가하는 감 잡기

Function	n					
	10	100	1,000	10,000	100,000	1,000,000
1	1	1	1	1	1	1
$\log_2 n$	3	6	9	13	16	19
n	10	10^2	10^3	10^4	10^5	10^6
$n * \log_2 n$	30	664	9,965	10^5	10^6	10^7
n^2	10^2	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

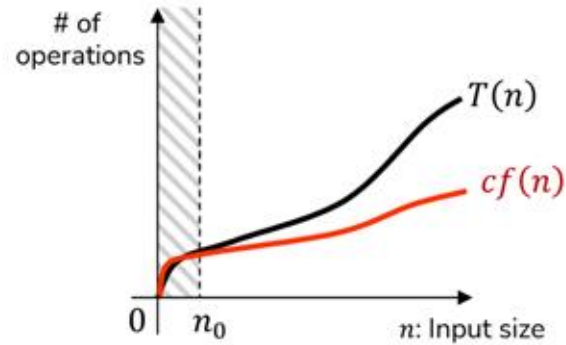


ASYMPTOTIC NOTATIONS

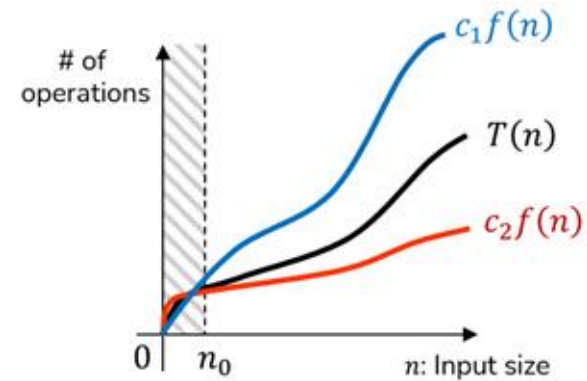
- 수학적으로 명확하게 정의하기 위한 점근적 표기법
- Big-O, Big-Omega, Big-Theta



Big-O



Big-Omega



Big-Theta



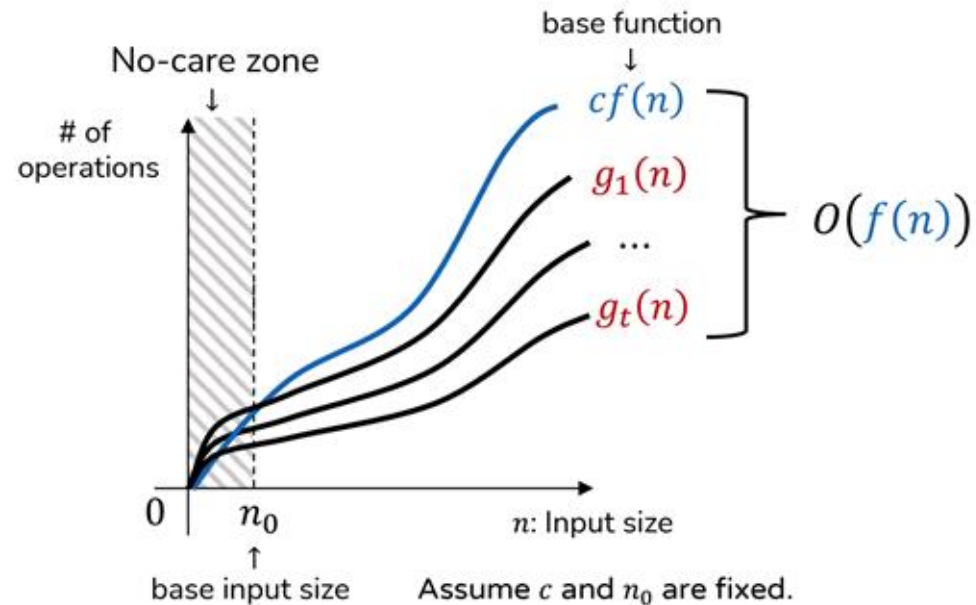
BIG-O NOTATION

- $O(f(n))$ 의 정의

$$O(f(n)) = \{g(n) \mid n \geq n_0 \text{인 모든 } n \text{에 대하여 } g(n) \leq c \cdot f(n) \text{인 양의 상수 } n_0 \text{와 } c \text{가 존재}\}$$

→ n 이 아주 클 때 $c \cdot f(n)$ 보다 값이 작은 함수의 집합

→ 상한 (Upper bound)



BIG-O NOTATION

- $O(f(n))$ 의 정의

$O(f(n)) = \{g(n) \mid n \geq n_0 \text{인 모든 } n \text{에 대하여}$
 $g(n) \leq c \cdot f(n) \text{인 양의 상수 } n_0 \text{와 } c \text{가 존재}\}$

- $T(n)$ 이 $O(f(n))$ 에 속하는지 증명

→ $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$ 를 만족하는 c 와 n_0 을 보이면 됨

Ex)

- $T(n) = 5n^2 \in O(n^2)$
- $T(n) = 4 \in O(1)$
- $T(n) = 3n^2 + 100 \in O(n^2)$
- $T(n) = n^2 \notin O(n)$



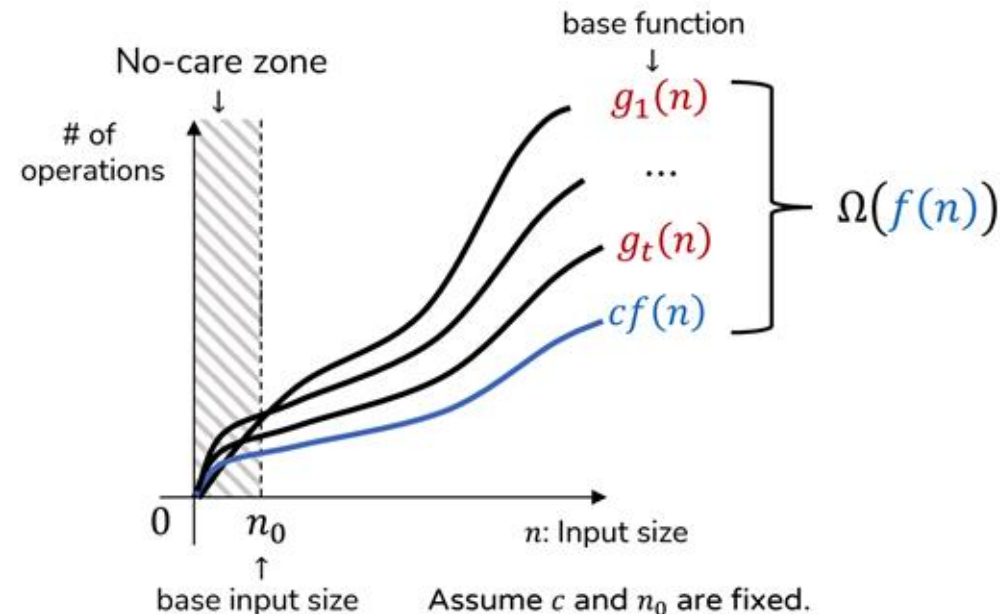
BIG-OMEGA NOTATION

- $\Omega(f(n))$ 의 정의

$$\Omega(f(n)) = \{g(n) \mid n \geq n_0 \text{인 모든 } n \text{에 대하여} \\ g(n) \geq c \cdot f(n) \text{인 양의 상수 } n_0 \text{와 } c \text{가 존재}\}$$

→ n 이 아주 클 때 $c \cdot f(n)$ 보다 값이 큰 함수의 집합

→ 하한 (Lower bound)



BIG-OMEGA NOTATION

- $\Omega(f(n))$ 의 정의

$\Omega(f(n)) = \{g(n) \mid n \geq n_0 \text{인 모든 } n \text{에 대하여}$
 $g(n) \geq c \cdot f(n) \text{인 양의 상수 } n_0 \text{와 } c \text{가 존재}\}$

- $T(n)$ 이 $\Omega(f(n))$ 에 속하는지 증명

→ $T(n) \geq c \cdot f(n)$ for all $n \geq n_0$ 를 만족하는 c 와 n_0 을 보이면 됨

Ex)

- $T(n) = 5n^2 \in \Omega(n^2)$
- $T(n) = 3n^2 + 100 \in \Omega(n^2)$
- $T(n) = n \notin \Omega(n^2)$

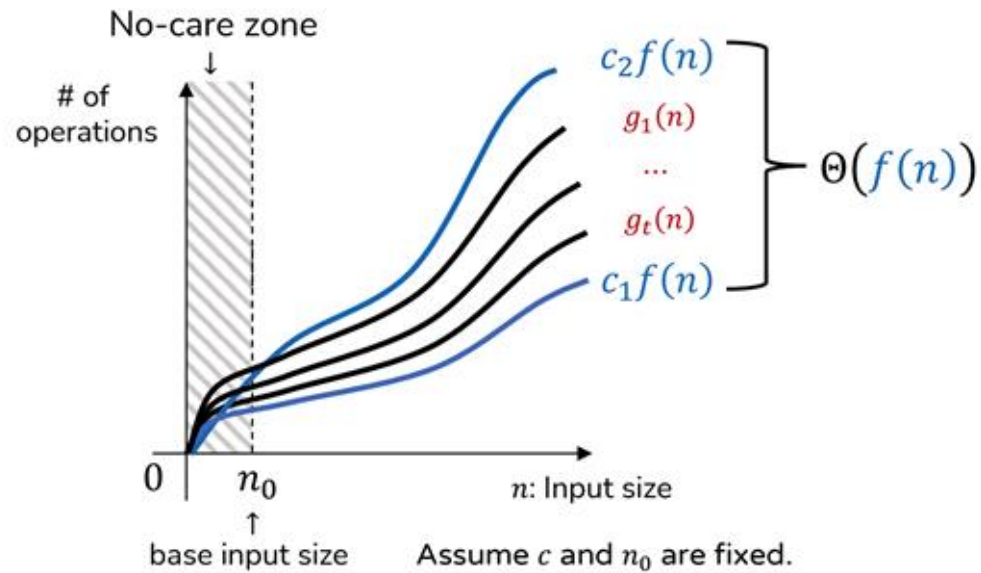


BIG-THETA NOTATION

- $\Theta(f(n))$ 의 정의

$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

➔ 상한과 하한이 모두 $f(n)$ 인 함수 집합



BIG-THETA NOTATION

- $\Theta(f(n))$ 의 정의

$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

- $T(n)$ 이 $\Theta(f(n))$ 에 속하는지 증명

→ $T(n)$ 이 $O(f(n))$ 과 $\Omega(f(n))$ 각각에 속하는지 증명하면 됨

Ex)

- $T(n) = 5n^2 \in \Theta(n^2)$
- $T(n) = 4 \in \Theta(1)$
- $T(n) = 3n^2 + 100 \in \Theta(n^2)$
- $T(n) = n \notin \Theta(n^2)$
- $T(n) = n^2 \notin \Theta(n)$



ASYMPTOTIC NOTATIONS

- Lower bound와 Upper bound는 Tight하게 잡아야 함
- Tight하지 않은 예시

	Algorithm 1	Algorithm 2	Algorithm 3
$T(n)$	$2n+1$	$2n^2+1$	2
점근적 분석	n	n^2	1
Big-O	$O(n), O(n^2), \dots$	$O(n^2), O(n^3), \dots$	$O(1), O(n), \dots$
Big-Omega	$\Omega(n), \Omega(1)$	$\Omega(n^2), \Omega(n), \dots$	$\Omega(1)$
Big-Theta	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$



PRACTICE

Ex1)

```
a = b;
```

Ex2)

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += n;
```

Ex3)

```
for (j=1; j<=n; j++)  
    for (i=1; i<=j; i++)  
        sum++;  
for (k=0; k<n; k++)  
    A[k] = k;
```



An abstract graphic on a light gray background. Two thin, dark gray lines intersect. One line is nearly vertical, starting from the top center and extending towards the bottom right. The other line is nearly horizontal, starting from the top left and extending towards the middle right. The word "SUMMARY" is written in a bold, black, sans-serif font, positioned to the right of the intersection point.

SUMMARY

SUMMARY

1. Algorithm analysis – Why? How?
2. Best / Worse / Average case
3. Asymptotic analysis
(Big-O, Big-Omega, Big-Theta)



NEXT LECTURE

Recurrence (재귀, 점화식)



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?